

TITLE AND DATE

Document name: CMPE311 Project 1

Document reference: provost.cmpe311.fall26.project#1

Date of publication: September 22, 2026

Lead engineer: Raheem Provost, UMBC

STAKEHOLDERS

Prof Kidd, Instructor, University of Maryland Bal2more County, Bal2more, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Bal2more County, Bal2more, Maryland, USA

HIGH-LEVEL DESCRIPTION

This document is the project report for Project #1 of the CMPE311 class. It contains the customer requirements, the derived high-level technical and educational requirements, the testing requirements, the system design, and the results of validation testing.

DESCRIPTION

This design implements an embedded system on an Arduino Uno R3 development platform that independently controls the blink rate of two LEDs. The User interacts with the System through the Arduino IDE serial monitor, specifying an LED number and a blink interval in milliseconds; the System then blinks that LED at the specified rate without interrupting the blinking of the other LED. Both LEDs continue blinking asynchronously and independently until the User changes one of their intervals.

RESULT SUMMARY

The project was a success—the embedded system design met all testing and high-level requirements.

REFERENCES AND GLOSSARY

DEFINITIONS

“The User” – The person operating (not programming) the embedded system

“The System” – The embedded system being operated by The User

“The Customer” – The person(s) paying for the embedded system being designed and built

“The Developer” – The person(s) designing and building the System

“The Evaluator” – The person(s) that determine whether or not The System satisfies The Customer-requirements.

“The Customer-requirements” – The requirements defined by The Customer as satisfying The Contract.

“The HL-requirements” – The System’s high-level technical requirements derived from The Customer-requirements.

“The Requirements” – Context dependent; e.g. may refer to the HL-requirements or Customer-requirements

“The Educational-constraints” – The requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.

“The Company” – The organization The Customer has contracted with to build The System.

“The Contract” – The business document that legally binds The Company to provide some service or product to The Customer.

“serial-monitor” – The serial port used by the Arduino IDE to communicate with The User.

“The Reference-platform” – The configuration of The System used by The Developer to test and validate The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.

ACRONYMS AND ABBREVIATIONS

Arduino – An Italian open-source hardware and software company; also refers to a development board created by that company

arduino.h – Header for a library of convenience functions specific to the Arduino development platform

AVR – A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology

avr.h – Header including helper functions and definitions for using an AVR microcontroller

EC – Educational constraint

ELEGOO – A Chinese company that develops and markets 3D printers and accessories

gcc – Front end for the GNU Compiler Collection

GitHub – A widely used web-based hosting platform for Git, a distributed software version control system

HL – High-level

IDE – Integrated Development Environment

LED – Light Emitting Diode

ms / msec – Millisecond

REQUIREMENTS

CONVENTIONS

- **Must, shall or will** – the design must satisfy the requirement
- **May** – the design may satisfy the requirement but does not have to
- **Informative** – the following is description only and contains no requirements
- **Identifiers:** C# customer; HL.#[.] high-level; EC.#[.] educational constraint; T.#[.] testing

PART I – CUSTOMER REQUIREMENTS AND DERIVED HIGH-LEVEL REQUIREMENTS

Informative: The requirements in Part I are independent of Parts II and III. The HL identifiers in Table 2 apply to Part I only; Parts II and III use the HL identifiers given in Table 3.

Table 1. Customer requirements (given)

ID	Customer requirement
C1	The User will be able to choose the blink rate of the LED with a button.
C2	The User will choose between 4 rates plus off.
C3	The blink rates must be set by an operator prior to the system being started
C4	Permitted blink rates must be perceivable by the viewer
C5	The minimum blink rate must be no slower than once every 5 seconds.
C6	The System must run upon an Arduino Uno R3 compatible board.

Table 2. High-level technical requirements derived from the customer requirements in Table 1

ID	High-level technical requirement	Derived from
HL.1	The System must include a single momentary pushbutton, connected to a digital input pin, that the User presses to select the LED's blink rate.	C1
HL.2	Each button press must advance the LED through a fixed cycle of exactly 5 states — OFF and 4 distinct blink rates — wrapping back to OFF after the 4th rate.	C2
HL.3	The button input must only be read, and the blink rate only take effect, before the System enters its run state (e.g., during a startup/configuration phase); once running, further button presses must have no effect until the System is reset or restarted.	C3

ID	High-level technical requirement	Derived from
HL.4	Each of the 4 permitted blink rates must have an on-time and off-time no shorter than 100 ms, so that the on/off transition is visibly perceivable rather than appearing as a flicker or steady glow.	C4
HL.5	The slowest of the 4 permitted blink rates must complete one full on-off cycle in 5000 ms or less.	C5
HL.6	The System must be implemented and must execute on an Arduino Uno R3 or a pin- and instruction-set-compatible clone (e.g., ATmega328P-based board such as the ELEGOO Uno R3).	C6

PART II – HIGH-LEVEL REQUIREMENTS AND EDUCATIONAL CONSTRAINTS

Informative: Parts II and III use the requirements below (given). The testing requirements derived from them are in the Testing and Validation section.

Table 3. High-level technical requirements for design and testing (given)

ID	High-level technical requirement
HL.1	The System must use at least 2 LEDs
HL.2	The System must use a standard Arduino compatible development board (e.g. the provided ELEGOO Uno R3)
HL.3	The System must communicate with The User via the Arduino IDE serial-monitor port
HL.4	Any use of the serial-monitor in HL.3 must be asynchronous and not affect the blinking of the LEDs.
HL.5	The User must set the blink interval of the LEDs in msec
HL.6	The blink rate of each LED must be constant unless changed by The User

Table 4. Educational constraints (given)

ID	Educational constraint
EC.1	The Developer must use the version of gcc used by Arduino IDE 2.3.3 or later for the AVR ATmega328p to cross compile any program used by The System
EC.2	The Developer must have any software used in The System hosted on a GitHub repository. This software does not need to include libraries, macros or other code not developed by The Developer.
EC.2.1	The Evaluator must be able to (a) download from the repository any software, documentation and other material needed for their evaluation, (b) cross compile any software, and (c) run that software on The Reference Platform.
EC.3	LEDs must be connected to Arduino digital Pins 2 & 3.
EC.4	The Evaluator must be able to successfully have The System perform the dialog in Table 7.
EC.5	The Developer must do bare metal programming, i.e. they cannot use any 3rd party multi-tasking libraries or programs such as FreeRTOS.

DESIGN

DESIGN PRE-REQUISITES

1. ELEGOO Arduino Uno R3
2. Arduino IDE 2.3.3 or later, with the AVR board package installed for the ATmega328P

BILL OF MATERIALS

Table 5. Bill of materials

Item	Qty	Value / part number	Purpose
Microcontroller board	1	Arduino Uno R3 (or ELEGOO Uno R3 clone)	Runs the LED-blink control program and hosts the serial-monitor interface
LED	2	Standard 5mm red LED	Visual output — blink rate set independently per LED (HL.1, HL.5)
Resistor	2	330 Ω	Current-limiting resistor in series with each LED
Breadboard	1	Half-size or full-size solderless breadboard	Mounting point for LEDs and resistors, and shared ground rail
Jumper Wires	4	Male-to-male	2 connect Arduino digital pins 2 & 3 to the LED anodes (via resistors); 2 connect the breadboard ground rail to Arduino GND
USB Cable		USB 2.0 Type-A to Type-B	Connects the Arduino to the computer for programming, power, and the serial-monitor link

DEVELOPMENT PLATFORM

Development was done using Arduino IDE version 2.3.10 on Windows 11. The IDE uses the AVR-GCC toolchain bundled with the Arduino AVR Boards package version 1.8.7, which cross-compiles for the ATmega328P target (satisfying EC.1).

SYSTEM OVERVIEW

The User interacts with the System entirely through the Arduino IDE serial monitor. On startup, the System prompts the User for an LED number (1 or 2) and then for a blink interval in milliseconds. Once both values are entered, the System begins blinking the selected LED at that interval while continuing to

blink the other LED (if already configured) at its own independently-set rate. The main loop checks for new serial input and updates both LEDs' blink states on every pass, so serial communication and LED blinking run concurrently without either blocking the other.

HARDWARE DESIGN

Each LED is connected to an Arduino digital output pin (LED1 to pin 2, LED2 to pin 3) through a 330 Ω current-limiting resistor in series, with the cathode returned to the breadboard's common ground rail, which connects back to the Arduino's GND pin, as shown in Figure 1. The resistor value was selected using the standard LED series-resistor formula, $R = (V_{\text{supply}} - V_{\text{LED}}) / I_{\text{LED}}$: using the Arduino's 5V logic-high output, a typical red LED forward voltage of $\sim 2\text{V}$, and a target forward current of $\sim 9\text{ mA}$, $R = (5\text{V} - 2\text{V}) / 0.009\text{A} \approx 333\ \Omega$, so a 330 Ω resistor (the nearest standard value) was used, limiting the current to roughly 9 mA — safely below the typical 20 mA maximum rating for a standard 5mm LED while still producing a clearly visible blink.

Figure 1. Schematic

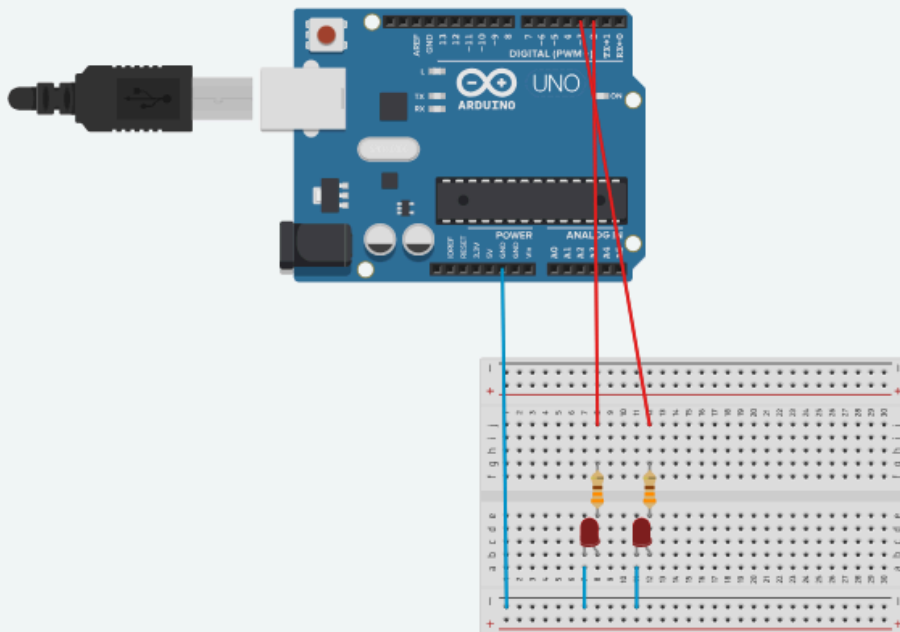


Table 6. Arduino pin assignments

Arduino pin	Signal / function	Connected to	Notes
Digital 2	Digital output — drives LED1	Anode of LED1 (via 330 Ω resistor)	Set HIGH/LOW in blinkLED1(); blink interval controlled by getInterval1
Digital 3	Digital output — drives LED2	Anode of LED2 (via 330 Ω resistor)	Set HIGH/LOW in blinkLED2(); blink interval controlled by getInterval2
GND	Common ground return	Breadboard ground rail, shared by both LED cathodes	Completes the current path for both LEDs

SOFTWARE DESIGN

The program uses the standard Arduino setup()/loop() structure with no blocking calls, so serial communication and LED blinking run concurrently, satisfying HL.4 and EC.5. Each LED has its own timestamp and state variables, so its blink timing is tracked and updated independently of the other. Serial input is read one character at a time and assembled into a line; once a full line (terminated by newline) is received, it's parsed as either an LED number or an interval, depending on which the System is currently waiting for. The entered interval is used as both the on-time and off-time, so a 600 ms interval produces a 600 ms on-phase and a 600 ms off-phase. Only 1 or 2 are accepted as valid LED numbers; any other value is ignored, and an LED does not begin blinking until it has been assigned a valid positive interval.

Repository URL: <https://github.com/raheempovost/CMPE311Project1>

TESTING AND VALIDATION

TESTING PLATFORM

1. ELEGOO Arduino Uno R3
2. Arduino IDE version 2.3.10
3. Powered through USB cable from the computer; LEDs connected directly to digital pins 2 and 3 through 330 Ω resistors, per Figure 1

REQUIRED TEST DIALOG

Table 7. Required test dialog (blue are the User inputs; given)

Serial port I/O	Notes
What LED? (1 or 2) 2	
What interval (in msec)? 600	LED2 starts blinking at an interval of 300ms on and 300ms off. LED1 is unaffected.
What LED? (1 or 2) 1	
What interval (in msec)? 1600	LED1 starts blinking at an interval of 800ms on and 800ms off. LED2 is unaffected.
What LED? (1 or 2)	Waiting for next LED# interval pair

TESTING AND VALIDATION REQUIREMENTS

Table 8. Testing and validation requirements

ID	Testing requirement	Verifies (HL / EC)
T.1	Both LEDs must blink, and each must keep blinking on its own until the user changes it.	HL.1, HL.6
T.2	The system must run on an Arduino Uno R3 or compatible board.	HL.2
T.3	The system must run the required serial monitor dialog (Table 7) without pausing or affecting either LED's blinking.	HL.3, HL.4, EC.4
T.4	The interval entered by the user (in ms) must match the LED's actual measured on/off time.	HL.5
T.5	The code must be on GitHub, built with the correct toolchain, and run on the Reference Platform.	EC.1, EC.2, EC.2.1
T.6	The LEDs must be wired to pins 2 & 3, and the code must not use any 3rd-party multitasking library.	EC.3, EC.5

REQUIREMENTS TRACEABILITY

Table 9. Requirements traceability — every requirement in Tables 3 and 4 is covered by at least one test (method: test, inspection, or demonstration)

Requirement	Verified by (T.#)	Method
HL.1 – At least 2 LEDs	T.1	Test
HL.2 – Standard Arduino-compatible board	T.2	Inspection
HL.3 – Communication via IDE serial monitor	T.3	Demonstration
HL.4 – Asynchronous serial use; blinking unaffected	T.3	Test
HL.5 – Blink interval set in msec	T.4	Test
HL.6 – Blink rate constant unless changed	T.1	Test
EC.1 – Built with IDE 2.3.3+ gcc (ATmega328P)	T.5	Inspection
EC.2 / EC.2.1 – On GitHub; Evaluator can build and run	T.5	Demonstration
EC.3 – LEDs on digital pins 2 & 3	T.6	Inspection
EC.4 – Required dialog (Table 7) works	T.3	Demonstration
EC.5 – Bare metal; no 3rd-party multitasking	T.6	Inspection

TEST PROCEDURES

Table 10. Test procedures

Test	Procedure (steps)	Expected result / pass criterion
T.1	Power the testbed with both LEDs wired to pins 2 and 3 per Figure 2. Upload the program. Enter an interval for LED1, then for LED2, and let both run without further input for at least 10 cycles each.	Both LEDs blink independently and continue blinking at a constant rate until changed.
T.2	Inspect the board used for the testbed and compare its markings/pinout to an Arduino Uno R3. Upload the program through the Arduino IDE and confirm it uploads without board-mismatch errors.	The board is an Uno R3 or a pin/instruction-set-compatible clone, and the IDE uploads to it successfully.
T.3	Open the serial monitor and run the dialog in Table 7, entering an interval for one LED while the other LED is already blinking. Observe both LEDs throughout.	The System prints each prompt and accepts input as shown in Table 7, and neither LED's blinking pauses or changes during entry.
T.4	Enter a specific interval (e.g., 600 ms) for one LED. Using a stopwatch or slow-motion video, measure the LED's actual on-time and off-time.	Measured on-time and off-time each equal the entered interval, within reasonable measurement tolerance.
T.5	Navigate to the project's GitHub repository. Download the source, compile it in Arduino IDE 2.3.3 (or later) targeting the ATmega328P, and upload it to the Reference Platform.	The repository contains the working source; it compiles without errors and runs correctly once uploaded.
T.6	Visually inspect the breadboard wiring against Figure 2/Table 6. Review the full source code for any <code>#include</code> of a multitasking library (e.g., FreeRTOS).	LEDs are wired to digital pins 2 and 3 only; no 3rd-party multitasking library appears anywhere in the source.

TESTING RESULTS

Table 11. Results of tests

Test	Result (Pass / Fail)	Data / observations
T.1	Pass	Both LEDs blinked independently and held their rate over 10+ cycles with no drift.
T.2	Pass	Board model/markings used and confirmation the sketch uploaded without errors.
T.3	Pass	Serial monitor transcript, and whether either LED paused or changed while the other was being set.
T.4	Pass	Entered interval vs. your actual measured on/off time, and the resulting error.
T.5	Pass	Repo URL, commit tested, and confirmation it compiled/ran cleanly.
T.6	Pass	Confirmation of exact pins used and that no multitasking library appears in your #includes.

APPENDIX A. TEST EVIDENCE

Table 12. Index of test evidence

Item	Description	File name / link	Supports test
Video	Both LEDs blinking independently at set intervals	https://github.com/raheempro/vost/CMPE311Project1/blob/main/cmpe311proj1vid.mp4	T.1
Screenshot	Successful IDE upload to the board	https://github.com/raheempro/vost/CMPE311Project1/blob/main/CMPE311proj1upload.png	T.2
Serial Monitor screenshot	Dialog from Table 7 running with both LEDs unaffected	https://github.com/raheempro/vost/CMPE311Project1/blob/main/CMPE311proj1SerialMonitor.png	T.3
Video	Measured on/off time vs. entered interval	https://github.com/raheempro/vost/CMPE311Project1/tree/main/Proof%20of%20measured%20time	T.4
Link	GitHub repository	https://github.com/raheempro/vost/CMPE311Project1	T.5
Code screenshot	Screenshot of pin setup (pinMode calls) and full #include list, showing no multitasking library used	https://github.com/raheempro/vost/CMPE311Project1/tree/main/Proof%20of%20no%20pin%20writing%20or%20multitasking	T.6

END OF DOCUMENT